

Woo_Matthew_Assignment01

Explanation:

twoSumBruteForce() find the two indices of the values that can add up to the target by going through each value and checking it with the values after it. For example, {2, 7, 11, 15} and target = 9, i = 0 and it runs through each one after it to see if they add up to the target.

twoSumHash() find the two indices of the values that can add up to the target by looking at the target and the current value and seeing what it needs. If it is not found, then put the current number as the key and the current index as the value in the map. In the next index, find a new needed by subtracting the target and the current nums[i] value. If needed is in the map, then return the current index and the index stored in the map.

Test/output evidence:

nums = {15, 4, 18, 8, 19, 22, 24, 59, 59, 20, 18, 12, 36, 42, 9}

Target: 24

twoSumBruteForce Test:

Indices: [0, 14]

Values: [15, 9]

$15 + 9 = 24$

This is valid

twoSumHash Test:

Indices: [1, 9]

Values: [4, 20]

$4 + 20 = 24$

This is valid

nums = {2, 7, 11, 15}

Target: 9

twoSumBruteForce Test:

Indices: [0, 1]

Values: [2, 7]

$$2 + 7 = 9$$

This is valid

twoSumHash Test:

Indices: [0, 1]

Values: [2, 7]

$$2 + 7 = 9$$

This is valid

nums = {1, 4, 6, 7, 11, 60}

Target: 61

twoSumBruteForce Test:

Indices: [0, 5]

Values: [1, 60]

$$1 + 60 = 61$$

This is valid

twoSumHash Test:

Indices: [0, 5]

Values: [1, 60]

$$1 + 60 = 61$$

This is valid

nums = {2, 4, 6, 10, 20, 24, 26, 50, 30}

Target: 50

twoSumBruteForce Test:

Indices: [4, 8]

Values: [20, 30]

$$20 + 30 = 50$$

This is valid

twoSumHash Test:

Indices: [5, 6]

Values: [24, 26]

$$24 + 26 = 50$$

This is valid

nums = {1, 12, 10, 11, 13, 17, 18, 20, 29}

Target: 30

twoSumBruteForce Test:

Indices: [0, 8]

Values: [1, 29]

$$1 + 29 = 30$$

This is valid

twoSumHash Test:

Indices: [4, 5]

Values: [13, 17]

$$13 + 17 = 30$$

This is valid

nums = {0, 10, 80, 60, 50, 30, 90, 100, 20}

Target: 100

twoSumBruteForce Test:

Indices: [0, 7]

Values: [0, 100]

$$0 + 100 = 100$$

This is valid

twoSumHash Test:

Indices: [1, 6]

Values: [10, 90]

$$10 + 90 = 100$$

This is valid

nums = {2, 7, 11, 15}

Target: 10

twoSumBruteForce Test:

No Solution

This is not valid

twoSumHash Test:

No Solution

This is not valid

Complexity discussion:

Brute Force version used two forloop, so the worst case is go through every value and solution is at the end. The time complexity is $O(n^2)$ since it is a nested forloop and the space complexity is $O(1)$ since it didn't take too much extra memory.

Hash version time complexity is $O(n)$ since the time it goes through the vector once. The space complexity is also $O(n)$ because the map can store up to the size of the vector.

Code:

```
// Name: Matthew Woo  
// Date: Sep 5, 2026  
// Class: CS 210  
// Professor: Dominic Dabish
```

```

/*
AI Usage Disclosure:
I used ChatGPT to help me understand and learn C++.
Nothing from this program were generated by AI.
The program structure and code provided in class were originally written by professor
Dabish.
Except for the class-provided code, the remaining implementation and work were
completed by myself.
*/

#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

pair<int,int> twoSumBruteForce(const vector<int>& nums, int target) {
    for (int i = 0; i < nums.size(); ++i) {
        for(int j = 0; j < nums.size(); ++j) {
            if (nums[i] + nums[j] == target) {
                return {i, j};
            }
        }
    }
    return {-1, -1}; // no solution found and -1 means can't find a solution
}

pair<int, int> twoSumHash(const vector<int>& nums, int target) {
    unordered_map<int, int> index;
    for (int i = 0; i < nums.size(); ++i) {
        int needed = target - nums[i];
        if (index.count(needed)) { // count() check if the key exists, if yes return
the indices
            return {index[needed], i}; // return the index from the map and the
current index
        }
        index[nums[i]] = i; // if it is not found, put the current number as the key
and current index as the value in the map
    }

    return{-1, -1}; // no solution found
}

//Function to test cases
void test(const vector<int>& nums, int target) { // void doesn't return anything
    //print the nums and target
    cout << "nums = {";
```

```

for (int i = 0; i < nums.size(); ++i) {
    cout << nums[i];
    if (i < nums.size() - 1) {
        cout << ", ";
    }
}
cout << "}\n";
cout << "Target: " << target << "\n";
//Test twoSumBruteForce Function from test cases
cout << "twoSumBruteForce Test:\n";
// variables that help make it easier to read
pair<int, int> answerBruteForce = twoSumBruteForce(nums, target);
int bruteForceIndex1 = answerBruteForce.first;
int bruteForceIndex2 = answerBruteForce.second;
int bruteForceValue1 = nums[bruteForceIndex1];
int bruteForceValue2 = nums[bruteForceIndex2];
// Print the indices and values at those indices
if(bruteForceIndex1 != -1 && bruteForceIndex2 != -1) {
    cout << "Indices: [" << bruteForceIndex1
        << ", " << bruteForceIndex2 << "]\n";
    cout << "Values: [" << bruteForceValue1
        << ", " << bruteForceValue2 << "]\n";
    cout << bruteForceValue1 << " + "
        << bruteForceValue2 << " = "
        << target;
}else{
    cout << "No Solution";
}
//Check if it is valid
if(bruteForceIndex1 != bruteForceIndex2 && bruteForceValue1 + bruteForceValue2 ==
target) {
    cout << "\nThis is valid";
}else{
    cout << "\nThis is not valid";
}

cout << "\n\n";

//Test twoSumHash Function from test cases
cout << "twoSumHash Test:\n";
// variables that help make it easier to read
pair<int, int> answerHash = twoSumHash(nums, target);
int hashIndex1 = answerHash.first;
int hashIndex2 = answerHash.second;
int hashValue1 = nums[hashIndex1];
int hashValue2 = nums[hashIndex2];
// Print the indices and values at those indices
if(hashIndex1 != -1 && hashIndex2 != -1) {

```

```

        cout << "Indices: [" << hashIndex1
            << ", " << hashIndex2 << "]\n";
        cout << "Values: [" << hashValue1
            << ", " << hashValue2 << "]\n";
        cout << hashValue1 << " + "
            << hashValue2 << " = "
            << target;
    }else{
        cout << "No Solution";
    }
    //Check if it is valid
    if(hashIndex1 != hashIndex2 && hashValue1 + hashValue2 == target) {
        cout << "\nThis is valid";
    }else{
        cout << "\nThis is not valid";
    }
    cout << "\n\n\n";
}

//Call function "test" in the main
int main() {
    //Required Test Case
    test({15, 4, 18, 8, 19, 22, 24, 59, 59, 20, 18, 12, 36, 42, 9}, 24);
    //Additional Test Case 1
    test({2, 7, 11, 15}, 9);
    //Additional Test Case 2
    test({1, 4, 6, 7, 11, 60}, 61);
    //Additional Test Case 3
    test({2, 4, 6, 10, 20, 24, 26, 50, 30}, 50);
    //Additional Test Case 4
    test({1, 12, 10, 11, 13, 17, 18, 20, 29}, 30);
    //Additional Test Case 5
    test({0, 10, 80, 60, 50, 30, 90, 100, 20}, 100);
    //Additional Test Case 6
    test({2, 7, 11, 15}, 10);
}

```